

WASM / REDIS / REACT

WasmRedis

Une base Redis lite, entierement dans le navigateur

Go en RAM. OPFS sur disque. React reste fluide.

React

SDK TS

Worker

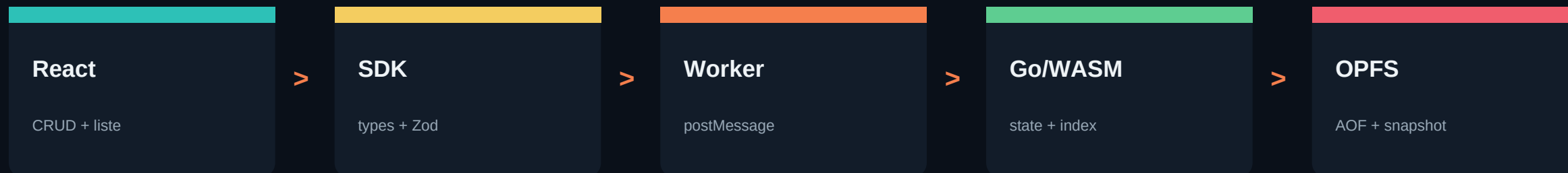
Go/WASM

OPFS

Une commande, cinq étapes

02

Le fil rouge de toute la presentation



Cycle de persistance



React ne bloque jamais : le moteur et le disque vivent dans le worker.

La RAM donne la vitesse ; AOF + snapshot donnent la persistance.

Le parser protege l'entree

03

Du texte vers une commande Go valide

`internal/redis/parser.go:44-51`

```
44     switch CommandType(commandName) {
45
46     // GET accepte soit une cle, soit la forme GET WHERE.
47     case CommandGet:
48         if len(parts) > 1 && strings.EqualFold(parts[1], "WHERE") {
49             return parseWhereCommand(parts)
50         }
51         return parseKeyCommand(CommandGet, parts)
```

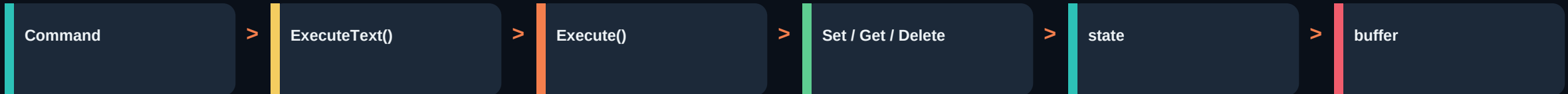
GET WHERE est distingue d'un GET par cle.

Toute commande inconnue est rejetee avant le moteur.

Du texte au state

04

ExecuteText relie le parser aux actions du moteur



internal/redis/engine.go:13-20

```
13 type Engine struct {
14     mu          sync.RWMutex
15     batchMu     sync.Mutex
16     bufferMu    sync.Mutex
17     state       Snapshot
18     buffer      []Operation
19     equalsIndex map[string]map[string]struct{}
20     numberIndex *BTree
```

internal/redis/engine.go:166-179

```
166 func (engine *Engine) ExecuteText(input string) (Result, []Operation) {
167     command, err := ParseCommand(input)
168     if err != nil {
169         return Result{OK: false, Error: err.Error()}, nil
170     }
171     return engine.Execute(command)
172 }
173
174 func (engine *Engine) Execute(command Command) (Result, []Operation) {
175     result, writes := engine.execute(command)
176     engine.appendToBuffer(writes)
177     return result, writes
178 }
179
```

Set, Get et Delete travaillent sur state.

Execute ajoute seulement les ecritures au buffer.

Du buffer au restore

05

AOF pour le journal, snapshot pour la photo complete



[internal/redis/engine.go:230-238](#)

```
230 func (engine *Engine) DrainBuffer() []Operation {
231     engine.bufferMu.Lock()
232     defer engine.bufferMu.Unlock()
233
234     writes := make([]Operation, len(engine.buffer))
235     copy(writes, engine.buffer)
236     engine.buffer = engine.buffer[:0]
237     return writes
238 }
```

DrainBuffer remet la file Go a zero sous mutex.

[web/src/worker/wasmRedis.worker.ts:356-367](#)

```
356 function saveSnapshot(): Promise<void> {
357     if (!scope.wasmRedisDumpSnapshot) {
358         return Promise.resolve();
359     }
360
361     return withStorageLock(async () => {
362         await writePendingAof();
363         const snapshot = scope.wasmRedisDumpSnapshot?.() ?? "{}";
364         await writeOpfsText(snapshotFileName, snapshot);
365         await writeOpfsText(aofFileName, "");
366     });
367 }
```

saveSnapshot ecrit le state puis compacte l'AOF.

Les plages passent par le B-Tree

06

Le parcours s'arrete des que la limite est depassee

[internal/redis/btree.go:95-103](#)

```
95     if operator == OperatorLessThan || operator == OperatorLessOrEqual {
96         tree.walkAscending(tree.root, func(item BTreeItem) bool {
97             if !matchesRange(item.Value, operator, target) {
98                 return false
99             }
100             items = append(items, item)
101             return true
102         })
103     return items
```

equals utilise un index inverse ; contains assume un scan.

Une plage selective reste a environ 0,3 us jusqu'a 10 000 entrees.

Le TTL supprime vraiment la cle

07

State, index et AOF restent cohérents

[internal/redis/engine.go:451-459](#)

```
451 func (engine *Engine) removeIfExpiredLocked(key string, stored StoredValue) bool {
452     if !engine.isExpired(stored) {
453         return false
454     }
455
456     delete(engine.state, key)
457     engine.removeFromEqualsIndexLocked(key, stored.Value)
458     engine.removeFromNumberIndexLocked(stored.Value)
459     return true
}
```

Expiration lazy au GET, plus balayage actif periodique.

Le restore garde ExpiresAt et ne remplit pas de nouveau le buffer.

React atteint le moteur Go

08

Le SDK et le bridge WASM transportent la commande



web/src/sdk/wasmRedis.ts:181-183

```
181     pending.set(id, { resolve, reject });
182     worker.postMessage(message);
183   });
```

cmd/wasm/main.go:66-87

```
66 func wasmRedisExecute(_ js.Value, args []js.Value) any {
69     if len(args) == 0 {
72         return encode(errorResponse{OK: false, Error: "missing request"})
73     }
76     var request executeRequest
79     if err := json.Unmarshal([]byte(args[0].String()), &request); err != nil {
82         return encode(errorResponse{OK: false, Error: err.Error()})
83     }
86     return encode(engine.ExecuteBatch(request.Commands))
87 }
```

Le SDK valide avec TypeScript et Zod avant l'envoi.

wasmRedisExecute decode puis appelle Engine.ExecuteBatch.

100 000 entrees, 29 lignes DOM

09

Virtual scroll maison + abonnement cible

[web/src/virtual/getVirtualRange.ts:22-27](#) | [web/src/store/entryStore.ts:60-64](#)

```
22   const firstVisible = Math.min(itemCount - 1, Math.floor(scrollTop / rowHeight));
23   const startIndex = Math.max(0, firstVisible - overscan);
24   const visibleCount = Math.ceil(viewportHeight / rowHeight) + overscan * 2;
25   const endIndex = Math.min(itemCount, startIndex + visibleCount);
26
27   return { startIndex, endIndex, totalHeight: itemCount * rowHeight };

60 export function useEntry(key: string): StoreEntry | undefined {
61   const subscribe = useCallback((listener: Listener) => subscribeEntry(key, listener), [key]);
62   const getSnapshot = useCallback(() => getEntry(key), [key]);
63   return useSyncExternalStore(subscribe, getSnapshot, getSnapshot);
64 }
```

Le spacer simule la hauteur totale ; seule la tranche visible est montee.

Modifier une ligne : compteur 1 -> 2 ; autre ligne : 1 -> 1.

Ce que la demo prouve

10

Mesures reelles, parcours reproductible

15,5 x

gain batch / 30 SET

~0,3 us

range selective / 10k

60 FPS

scroll mesure

29

lignes DOM / 100k

Demo en 60 secondes

- 1 SET avec TTL, puis Flush
- 2 Reload : restore OPFS
- 3 Seed 100, filtre ≥ 50
- 4 Editer : 1 -> 2
- 5 Lancer Benchmark

Validation finale

Go race tests	OK
Tests TypeScript	OK
Build WASM + React	OK